

Software Architecture

EE 8217/EC 8208



Based on Software Architecture in Practice (3rd Edition)
By Len Bass, Paul Clements, Rick Kazman

Outline

Architectural Patterns

- Few types of patterns and example

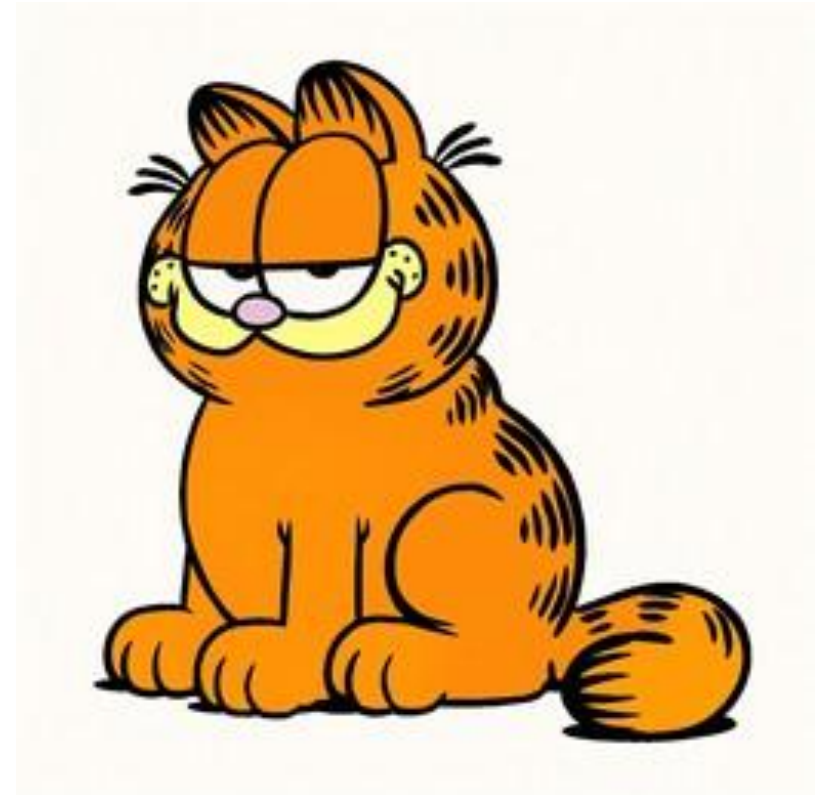
Design Concepts

- Abstraction
- Modularity
- Separation of Concerns
- Information Hiding
- Refactoring etc.



Activity

Define the role of a software architect. Do a brainstorm with your team.



Laws of Software Architecture

First Law of Software Architecture

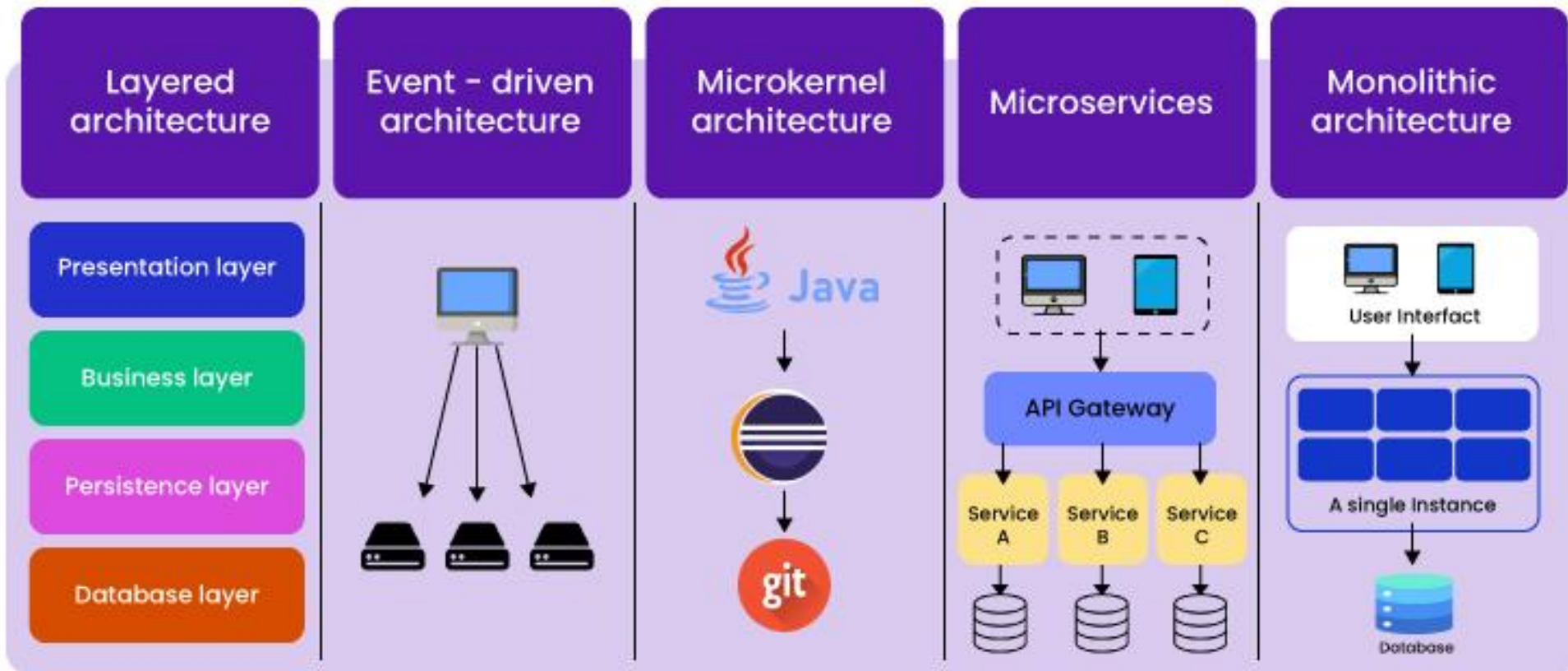
- Everything in software architecture is a trade-off -

Second Law of Software Architecture

- Why is more important than how -

Architectural Patterns

Top 5 Most – Used Architecture Patterns



Layered Architecture Pattern

- **Presentation layer**

The user interface layer where we see and enter data into an application.

- **Business layer**

This layer is responsible for executing business logic as per the request.

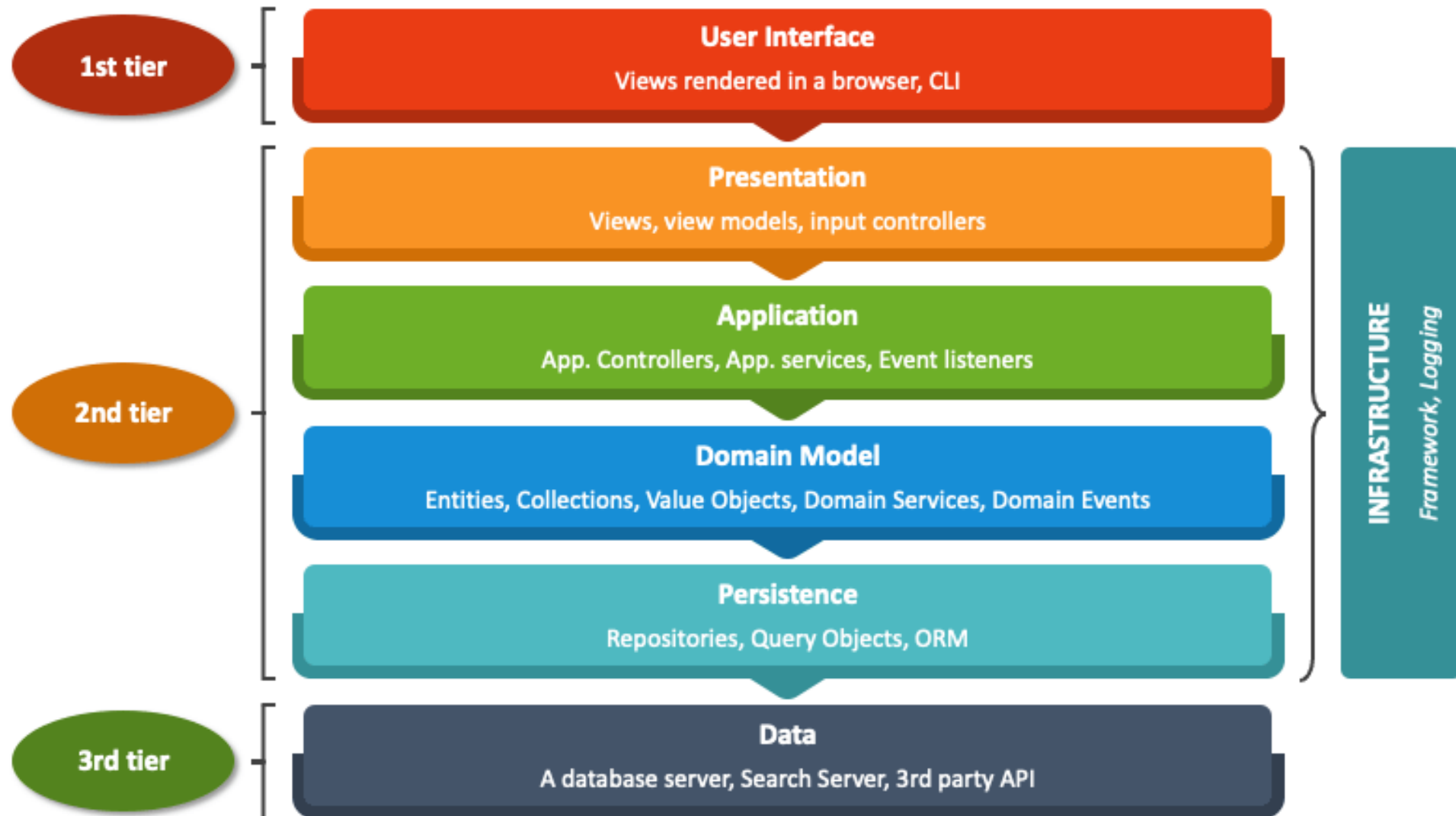
- **Application layer**

This layer acts as a medium for communication between the 'presentation layer' and 'data layer'.

- **Data layer**

This layer has a database for managing data.

LAYERED ARCHITECTURE



Layered Architecture– Use cases

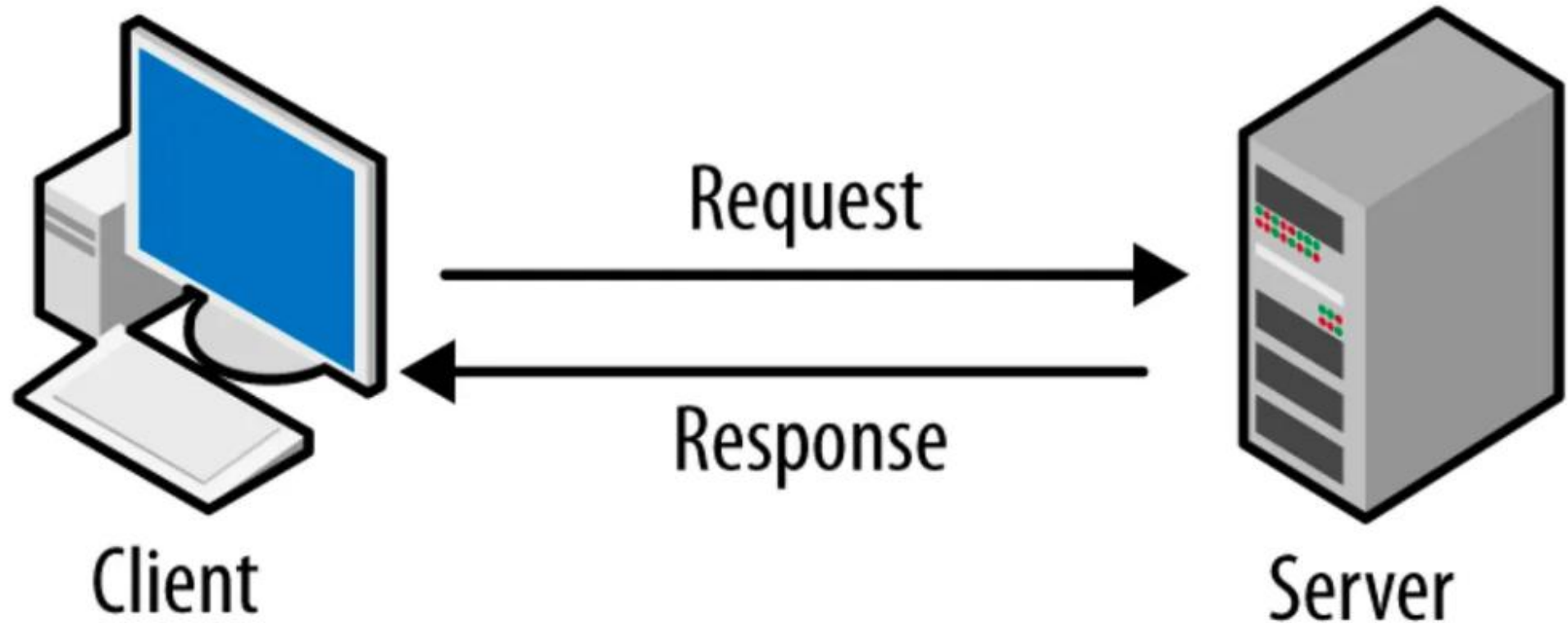
- Enterprise Applications like Customer Relationship Management (CRM).
- Web Applications like E-commerce platforms.
- Desktop Applications such as Financial Software.
- Mobile Applications like Banking applications.
- Content Management Systems like Wordpress.

Activity

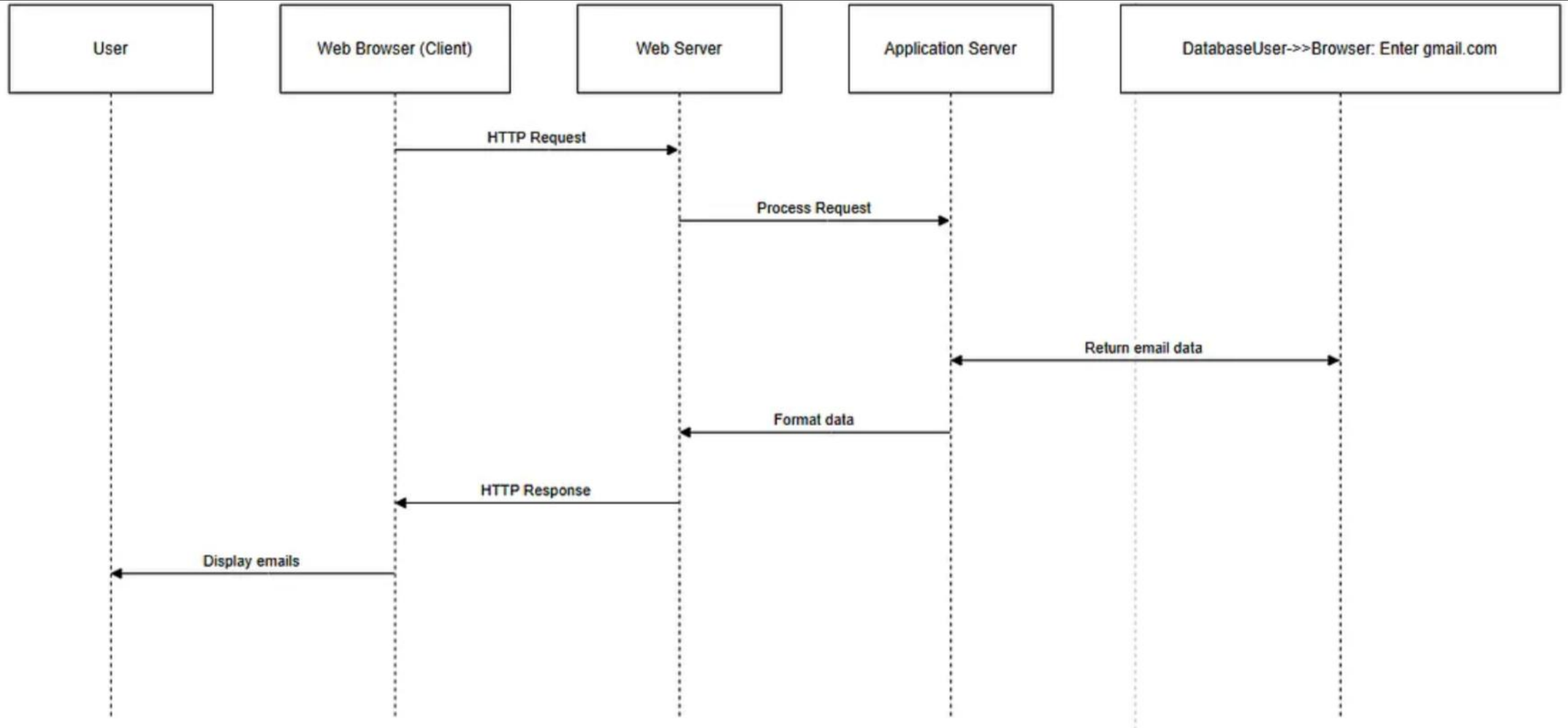
**Advantages of
Layered
Architecture?????**



Client-Server Architecture Pattern



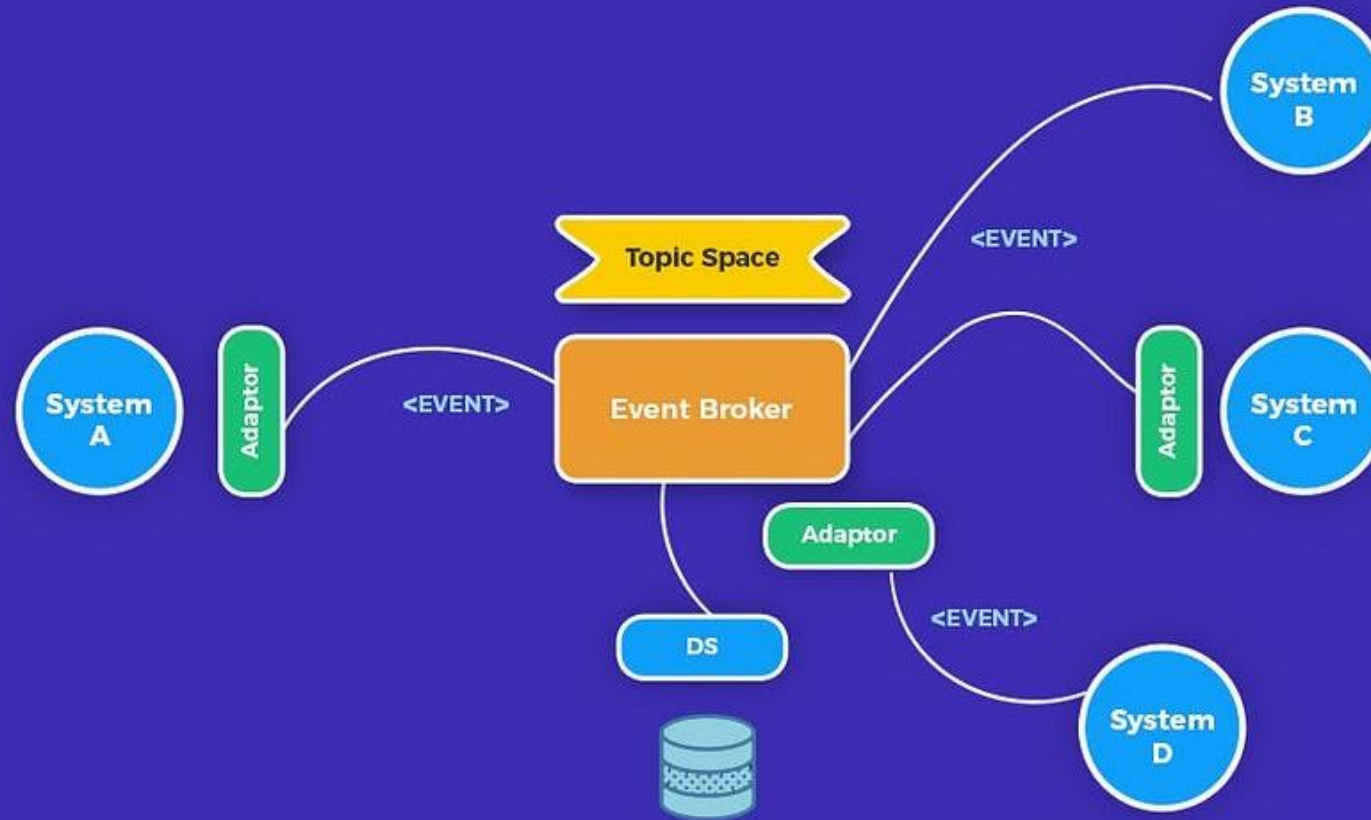
Me



variations of client-server architecture

- **Two-tier architecture**
- **Three-tier architecture**
- **N-tier or multi-tier architecture**

Event-Driven Architecture Pattern



EDA : Use Cases

- Real-Time Analytics like stock market analysis systems.
- IoT Applications like smart home systems.
- Financial Systems like fraud detection systems monitor transactions in real-time.
- Online multiplayer games.
- Customer Support Systems like Chatbots.

Microservices Architecture Pattern

- The collection of small services that are combined to form the actual application is the concept of microservices pattern.
- Instead of building a bigger application, small programs are built for every service (function) of an application independently.
- And those small programs are bundled together to be a full-fledged application.
- So adding new features and modifying existing microservices are no longer a challenge

Microservices Architecture Pattern: Use cases

- E-commerce Platforms:

Amazon and eBay

- Streaming services:

Netflix and Spotify

- Social Media Platforms :

Twitter and Facebook

- Online Banking Platforms.

Design Concepts

Abstraction

Modularity

Coupling and Cohesion

Separation of concerns

Information Hiding

Refactoring

Encapsulation

Abstraction

- It is the act of representing essential features without including the background details or explanations.
- Used to reduce complexity
- Allow efficient design and implementation of complex software systems.

highest level of abstraction??

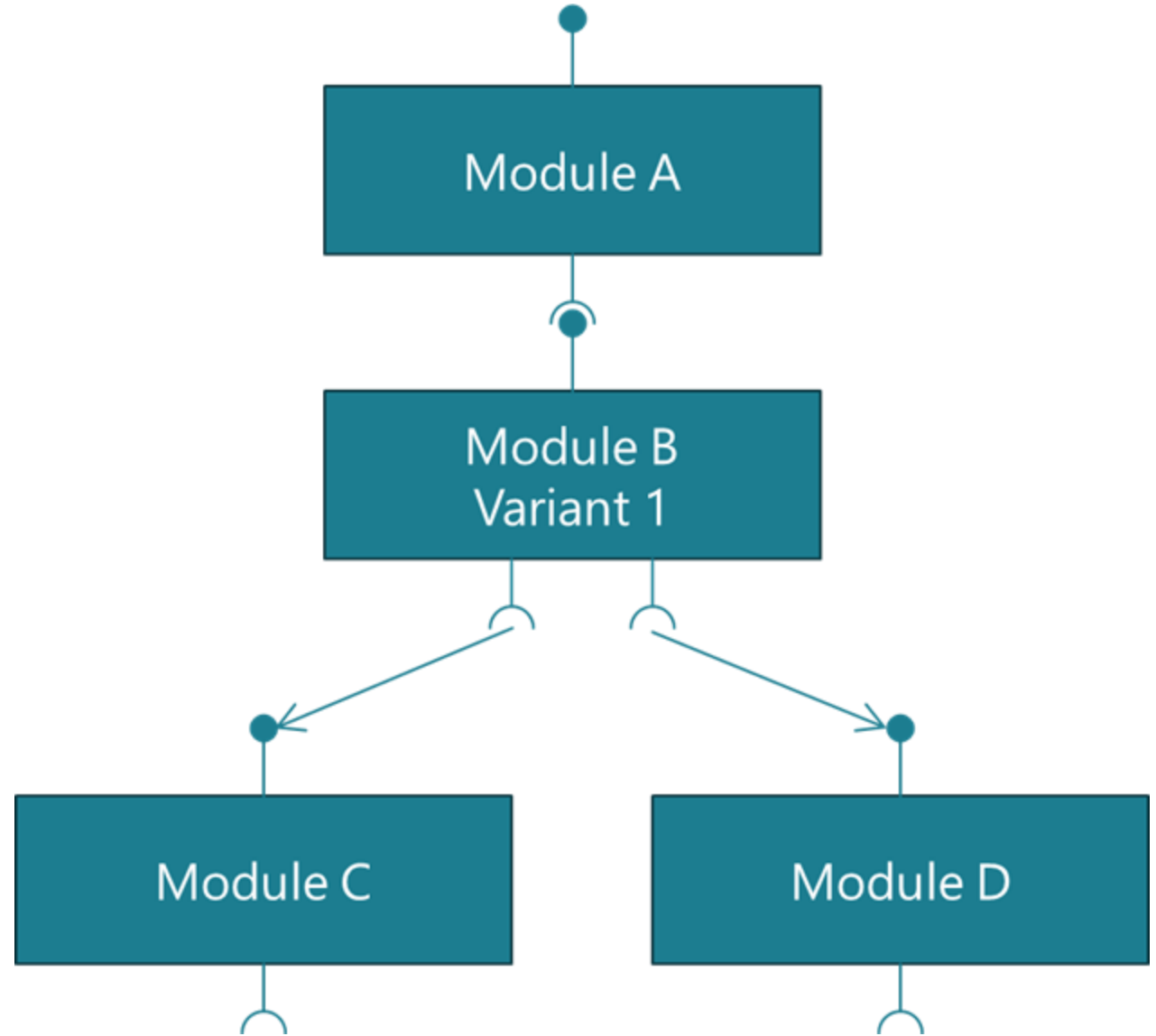
lower levels of abstraction??

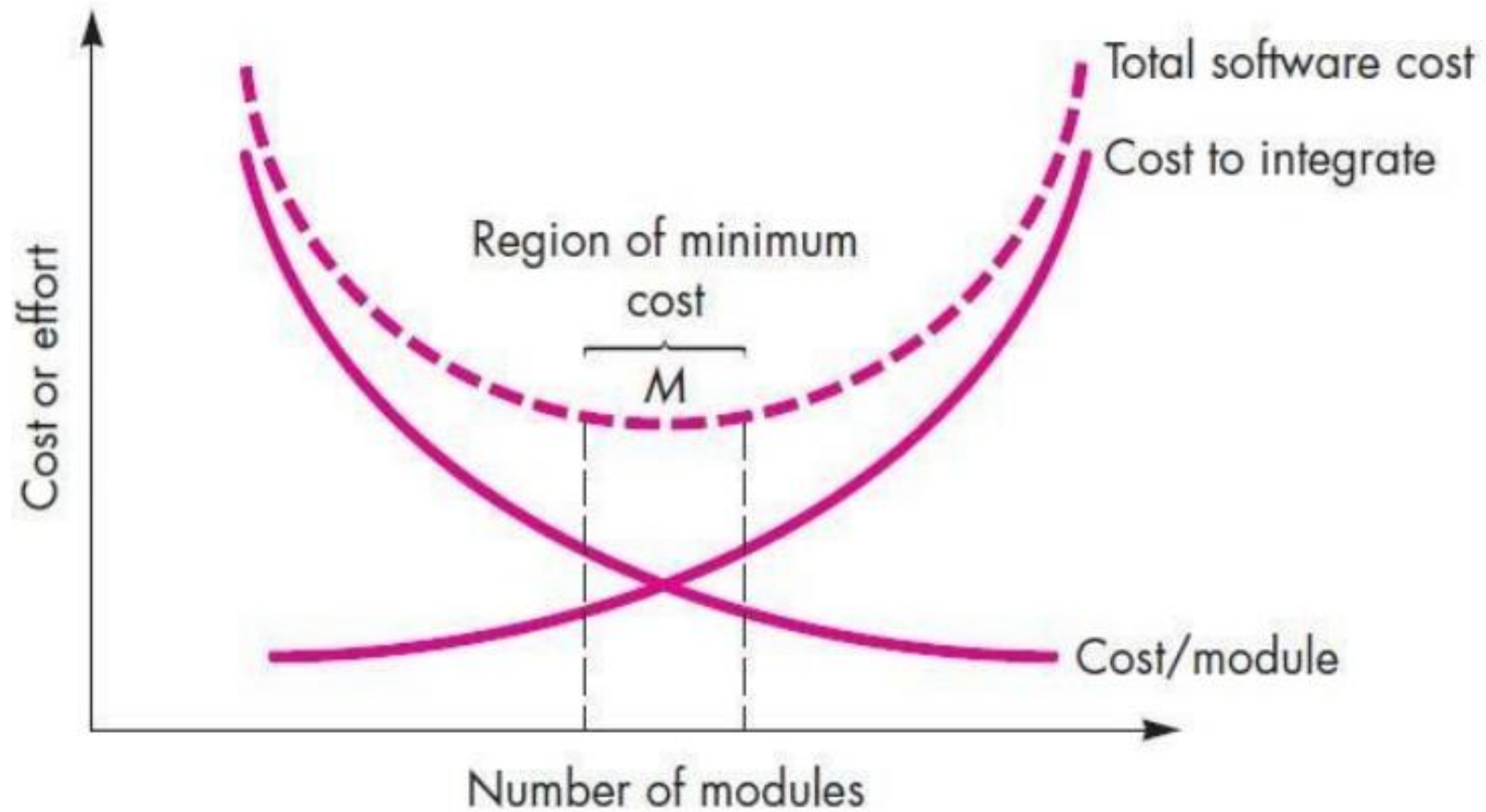
Example



Modularity

- Software is divided into separately named and addressable components, called module.





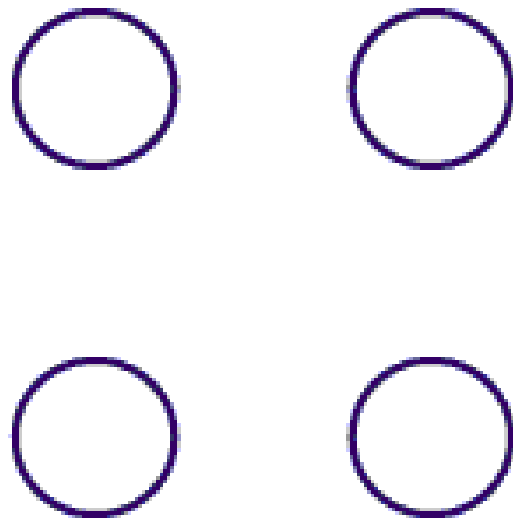
Example



Coupling and Cohesion

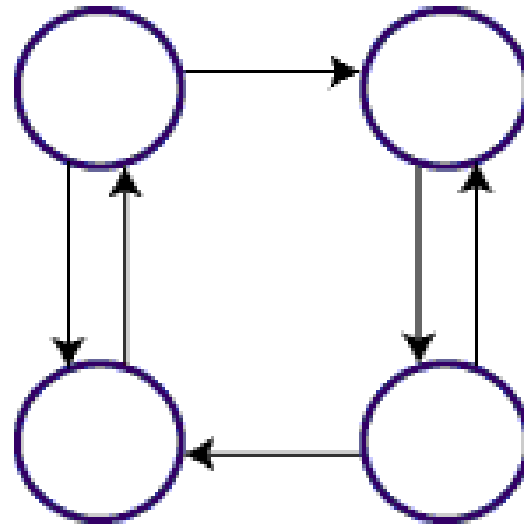
- **Coupling** is an indication of interconnection among modules in a software structure.
- Depends on the interface complexity between modules, the point at which entry or reference is made to a module, and what data pass across the interface.
- In software design, you should strive for the lowest possible coupling.

Module Coupling



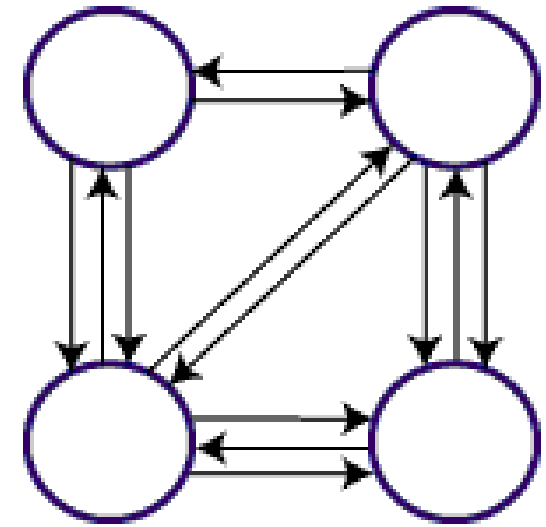
Uncoupled: no dependencies

(a)



Loosely Coupled: Some dependencies

(b)

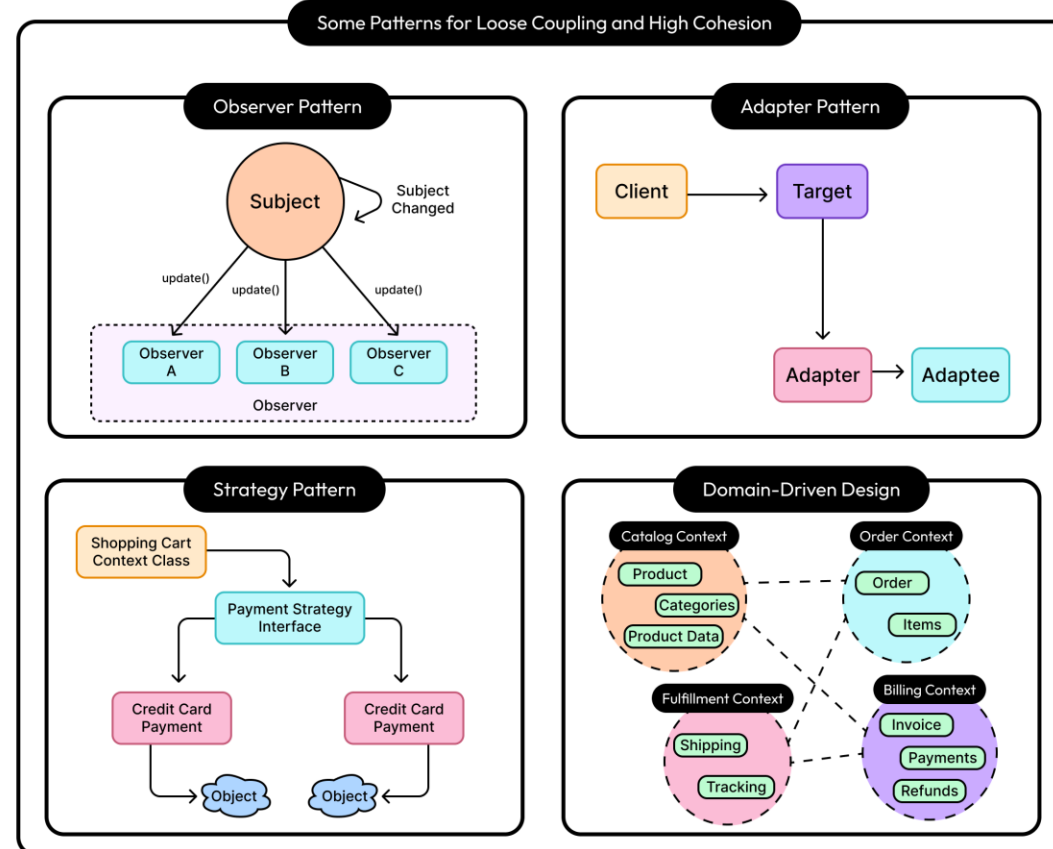
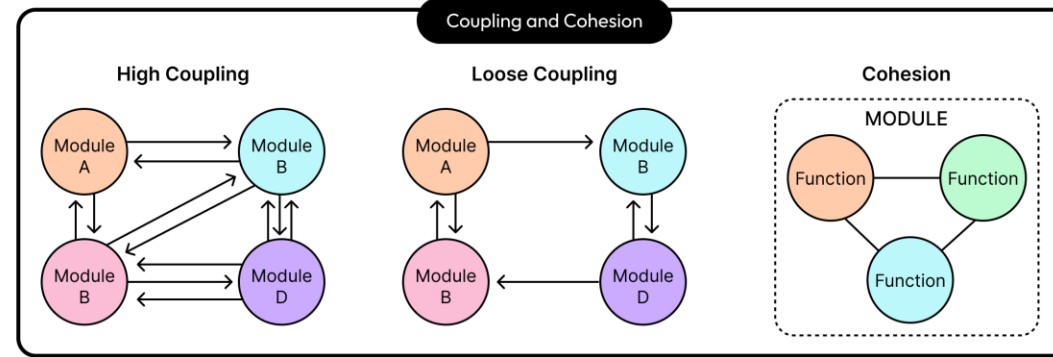


Highly Coupled: Many dependencies

(c)

- **Cohesion** is a natural extension of the information-hiding concept.
- A cohesive module performs a single task, requiring little interaction with other components in other parts of a program.
- Stated simply, a cohesive module should do just one thing. Although you should always strive for high cohesion

Coupling and Cohesion In Software Architecture



Separation of Concerns

- This suggests that any complex problem can be more easily handled if it is subdivided into pieces that can each be solved or optimized independently.
- A concern is a feature or behavior that is specified as part of the requirements model for the software

Example



Information Hiding

- suggests that modules be “characterized by design decisions that hides from all others.”
- modules should be specified and designed so that information contained within a module is inaccessible to other modules that have no need for such information.
- benefits when modifications are required during testing and later during software maintenance.

Example



Refactoring

- is a reorganization technique that simplifies the design (or code) of a component without changing its function or behavior.
- “Refactoring is the process of changing a software system in such a way that it does not alter the external behavior of the code [design] yet improves its internal structure.”

-Fowler-

Example

